19 April 2026

PRACTICAL

OBJECT ORIENTED LANGUAGE WITH JAVA

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GOVT. POLYTECHNIC COLLEGE PATIALA

Submitted By: Priyanka

Branch: Computer Science & Engineering

Semester: 4th

Submitted To:

Mr. Lalit Singh

Maan

INDEX OF PRACTICAL

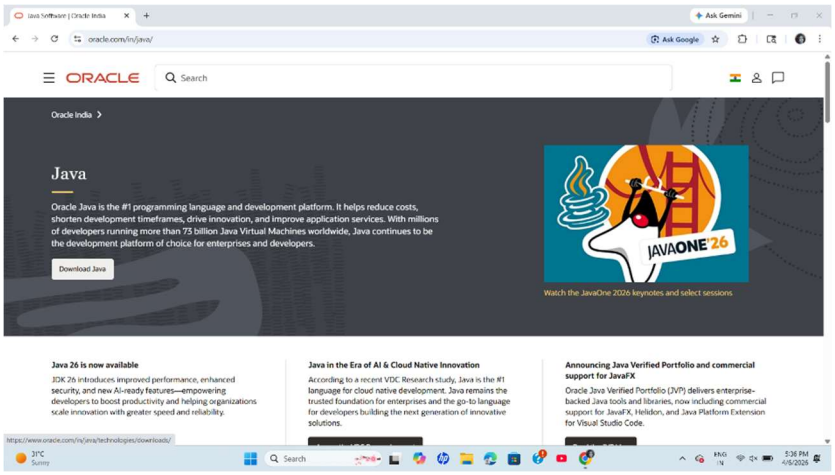
SR NO.	PRACTICAL NAME	REMARKS
1.	To download, install, and configure the Java Development Kit (JDK) on Windows.	
2.	Programming exercise on control flow statements, operators, and looping statements in Java.	
3.	Program to scan different types of input from the user using the Scanner class.	
4.	Programming exercise on arrays and functions in Java.	
5.	Program to demonstrate the concept of classes and objects using access specifiers.	
6.	Program to demonstrate the concept of classes and objects using access specifiers.	
7.	Programming exercise on different types of inheritance in Java.	
8.	Program to demonstrate the concept of method overloading and overriding.	
9.	Program to demonstrate the concept of packages, abstract classes, and interfaces.	
10.	Programming exercise on exception handling.	

PRACTICAL-1

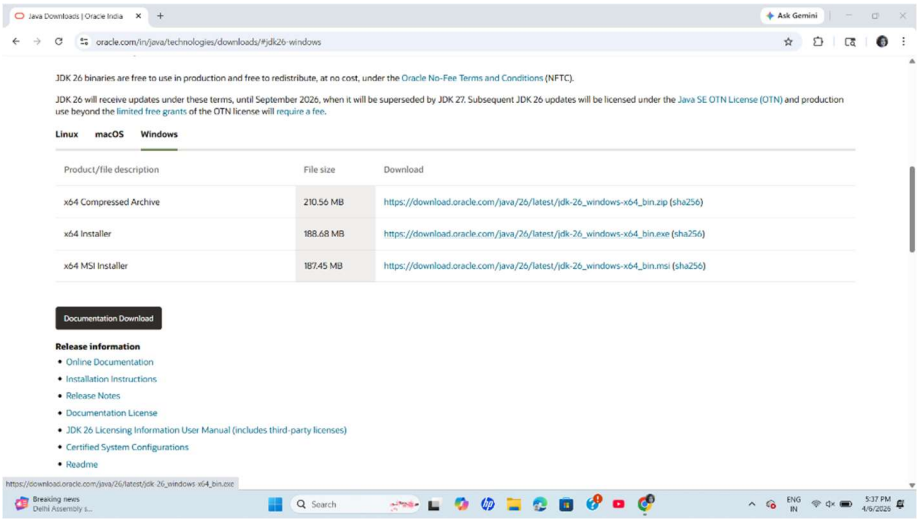
Aim: To download, install, and configure the Java Development Kit (JDK) on Windows.

Part 1: Downloading Java (JDK 26)

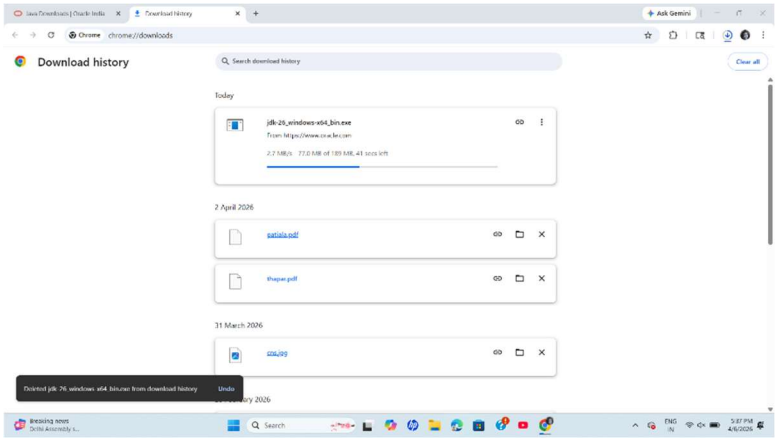
Step 1: Open your web browser and navigate to the official Oracle Java website.



Step 2: Scroll to the JDK 26 downloads section, select the **Windows** tab, and click the download link for the **x64 Installer**.



Step 3: Wait for the jdk-26_windows-x64_bin.exe setup file to finish downloading in your browser.



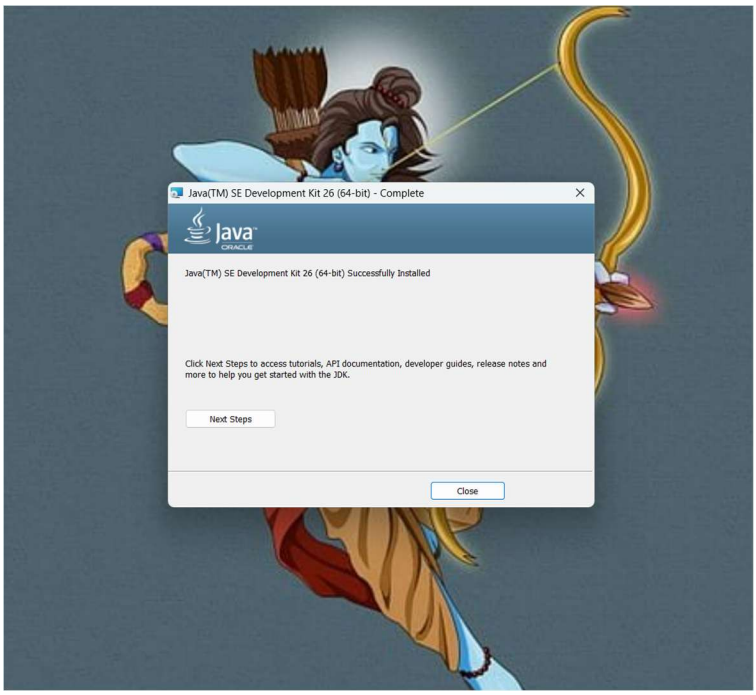
Part 2: Installing Java

Step 4: Open the downloaded .exe file to launch the Java SE Development Kit 26 Installation Wizard and click **Next**.

Step 5: Leave the destination folder as the default path (C:\Program Files\Java\jdk-26\) and click **Next**.

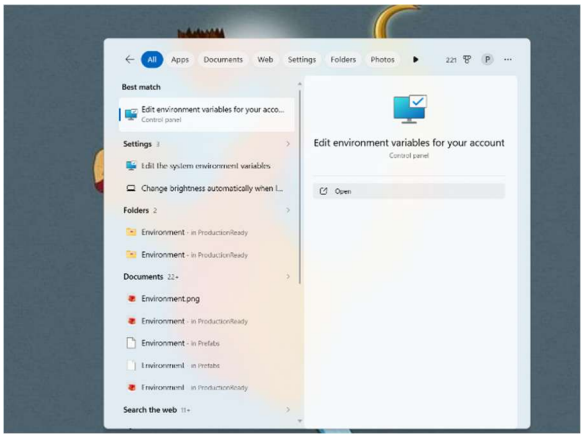
Step 6: Wait for the installation progress bar to fill as the installer extracts the necessary files.

Step 7: Once the "Successfully Installed" message appears, click **Close** to exit the wizard.

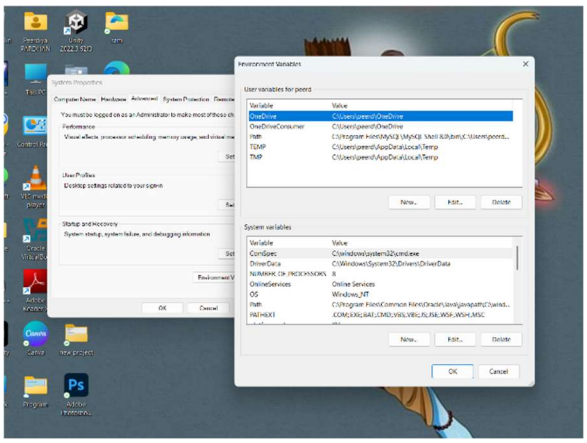


Part 3: Opening Environment Variables

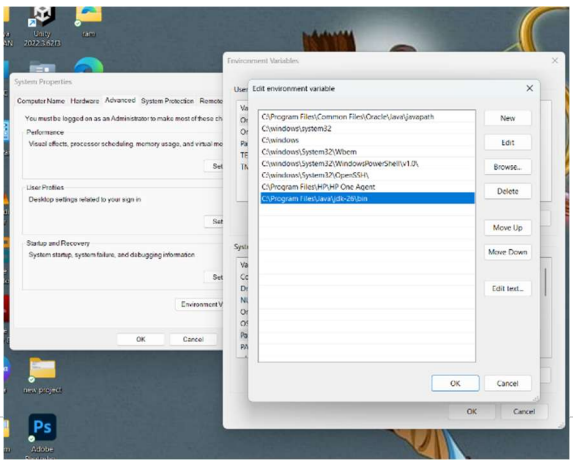
Step 8: Open the Windows Start menu, search for **"Edit environment variables for your account"**, and click to open the Control Panel setting.



Step 9: In the System Properties window, go to the **Advanced** tab and click the **Environment Variables...** button at the bottom.

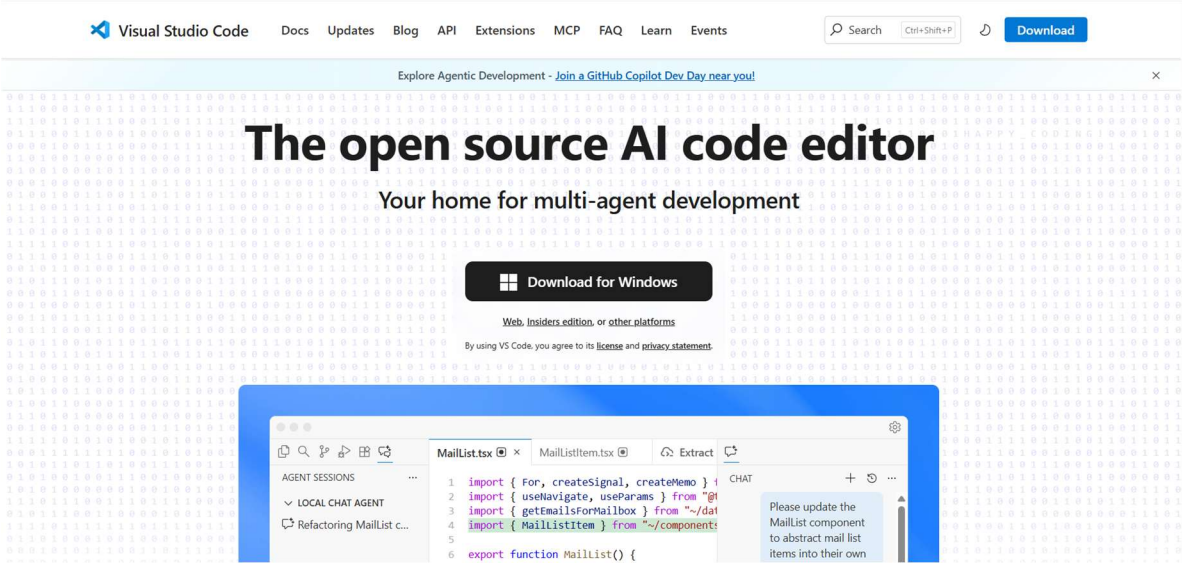


Step 10: The Environment Variables window will open, displaying your User and System variables. You will use this window to configure the Path for Java.

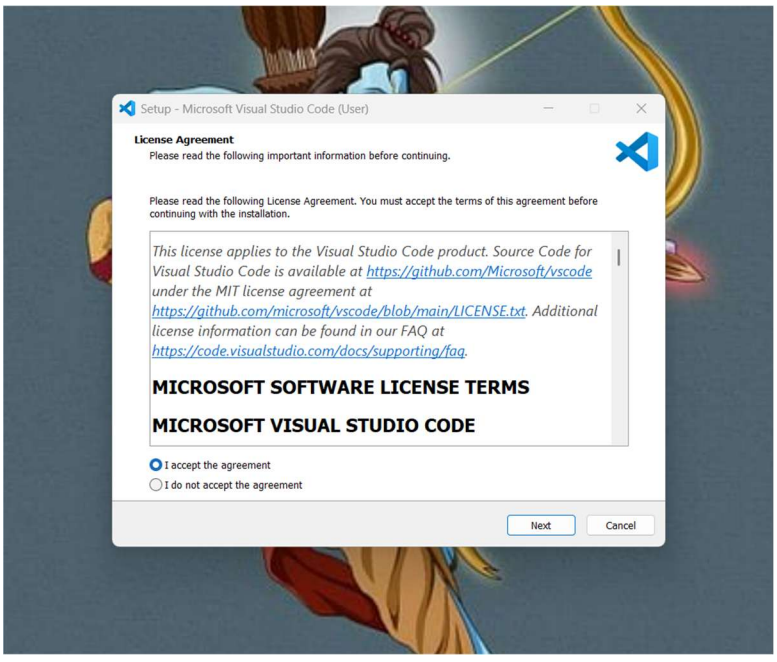


Part 4: Downloading Visual Studio Code

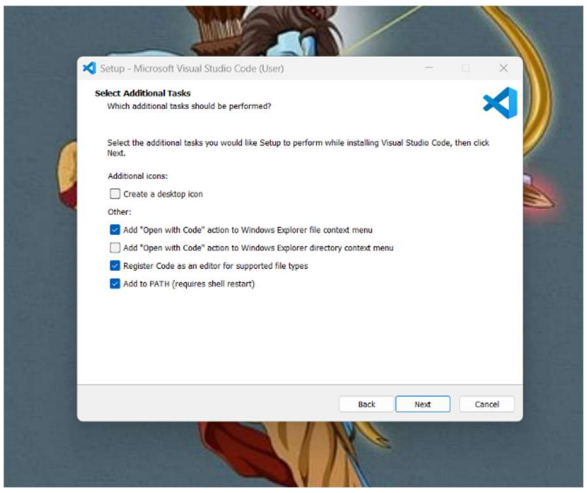
Step 11: Open your web browser, navigate to the official Visual Studio Code website (code.visualstudio.com), and click the large **Download for Windows** button.



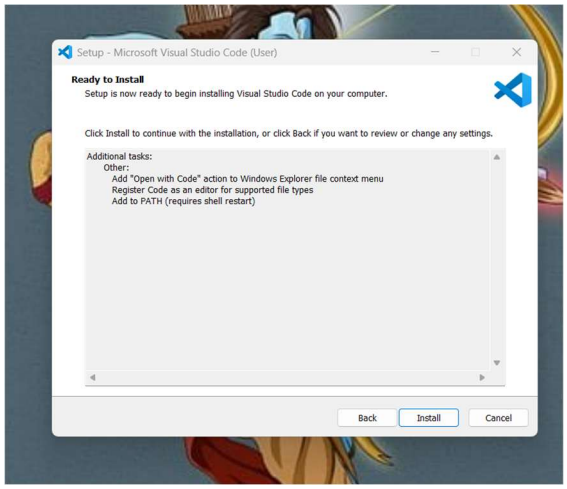
Step 12: Run the downloaded VS Code installer. On the License Agreement screen, select "I accept the agreement" and click **Next**.



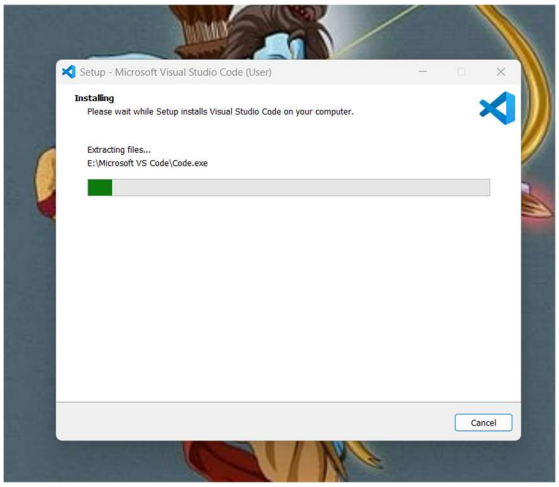
Step 13: Check the boxes to add the "Open with Code" action to your context menus and ensure "Add to PATH" is checked, then click **Next**.



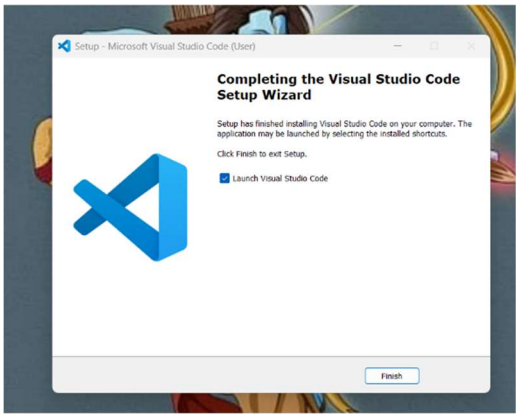
Step 14: Review your installation settings on the "Ready to Install" screen and click **Install**.



Step 15: Wait for the setup wizard to extract the files and complete the installation process.

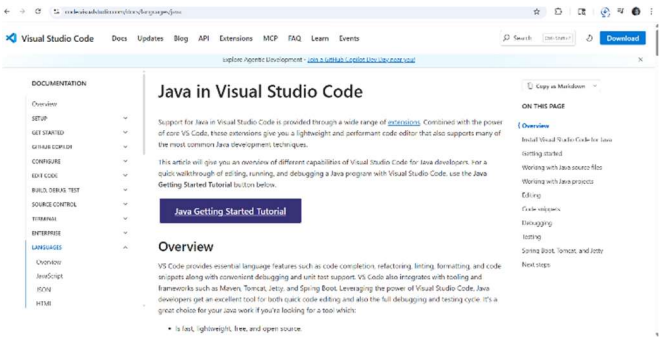


Step 16: Once finished, keep the "Launch Visual Studio Code" box checked and click **Finish**.

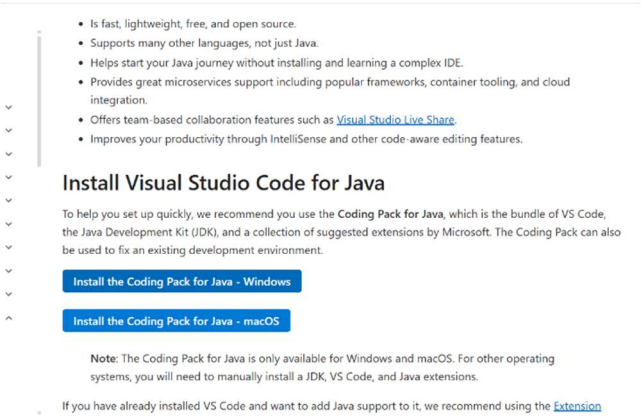


PART 5: Installing the Coding Pack for Java

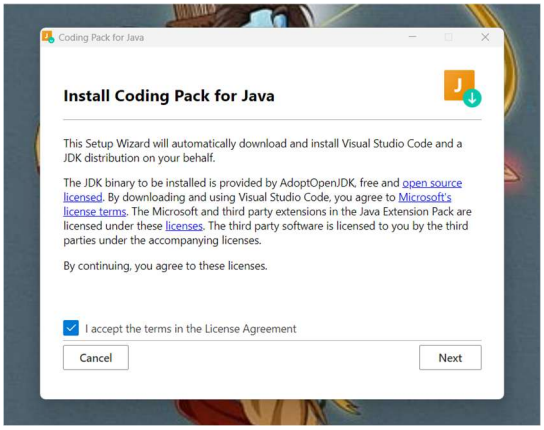
Step 17: Open your web browser and search for the official VS Code Java documentation. Navigate to the page titled "Java in Visual Studio Code."



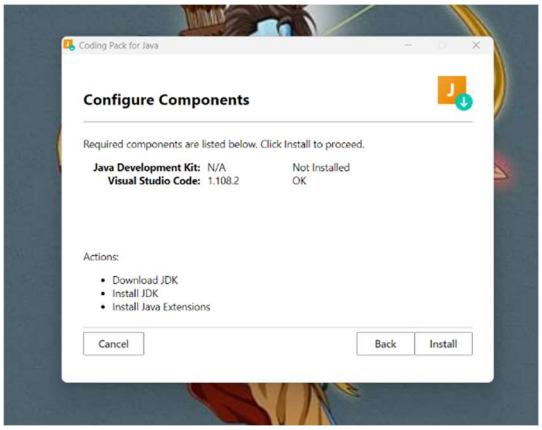
Step 18: Scroll down to the "Install Visual Studio Code for Java" section and click the button label **Install the Coding Pack for Java - Windows**.



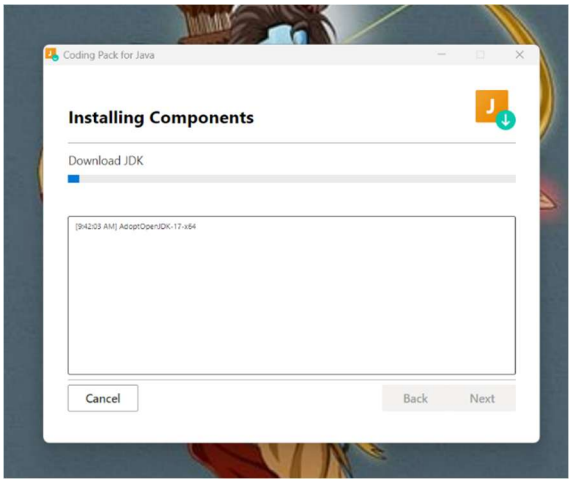
Step 19: Open the downloaded Coding Pack installer. Accept the License Agreement and click **Next**.



Step 20: The installer will automatically configure the components. Click **Install** to proceed.

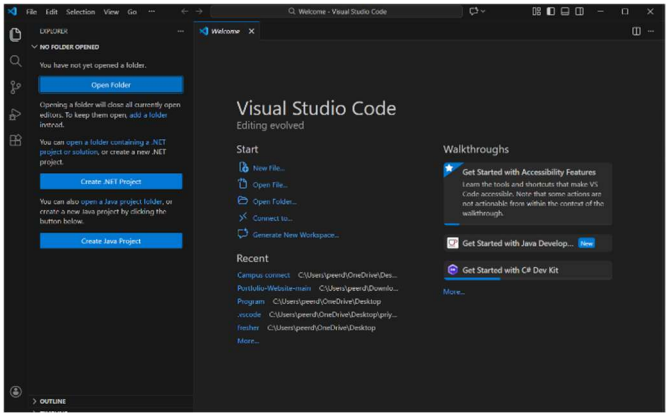


Step 21: The wizard will begin downloading and installing the required JDK components and Java extensions. Wait for this to complete.

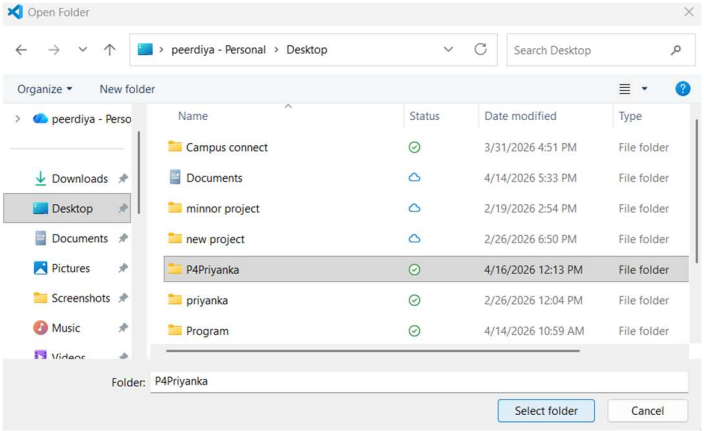


Part 6: Creating a Project and Running Java Code

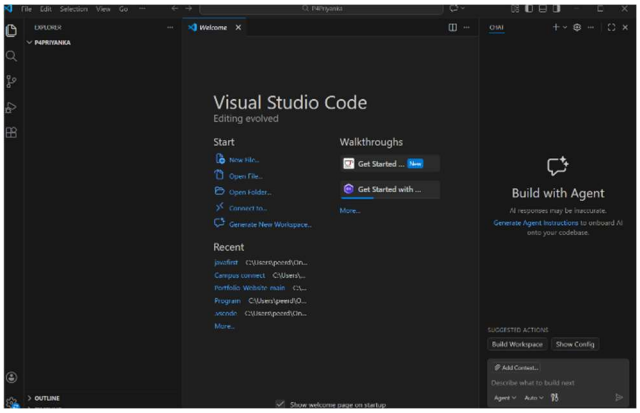
Step 22: Open Visual Studio Code. On the Welcome screen, click on the **Open Folder** button.



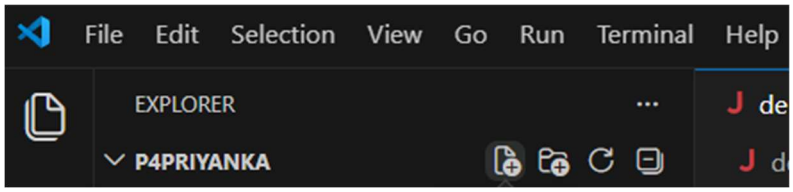
Step 23: A File Explorer window will open. Navigate to your Desktop, create a new folder for your project (e.g., P4Priyanka), select it, and click **Select Folder**.



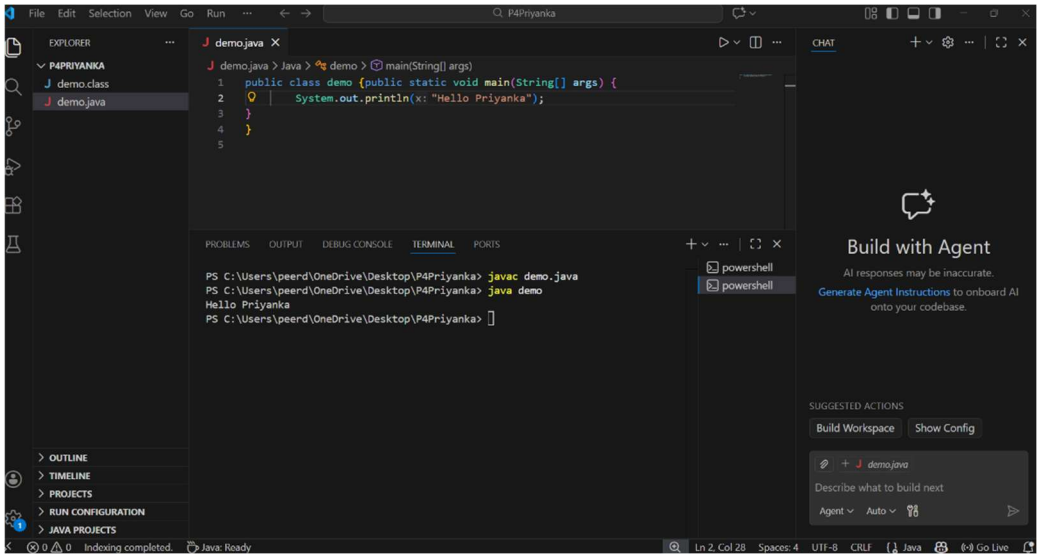
Step 24: The folder you selected will now be visible in the VS Code Explorer sidebar on the left, ready for you to add files.



Step 25: Create a new file (e.g., first.java) and write a basic print statement.



Step 26: To execute the code perfectly, create a file named demo.java and write your program, making sure the class name demo matches the file name. Open the Terminal at the bottom, type JAVAC demo.java to compile it, and then type java demo to run it. You will see your output (e.g., "Hello Priyanka") successfully printed in the terminal.



PRACTICAL-2

Aim: Programming exercise on control flow statements, operators, and looping statements in Java.

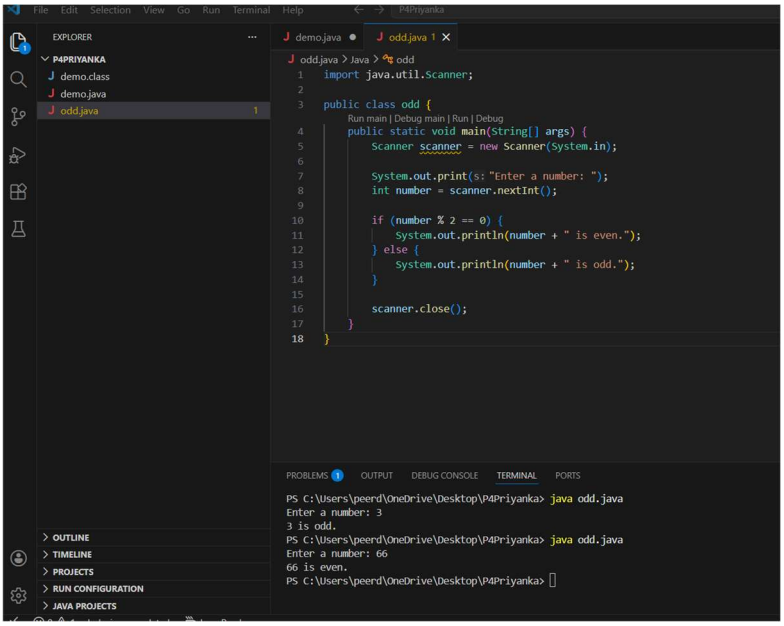
Control Flow Statement

Exercise 1: If - Else Statement Objective: Write a Java program that checks if a number is even or odd.

Input (Program - odd.java):

Java

```
import java. util. scanner;


public class odd {
   public static void main (String [] args) {
      Scanner scanner= new Scanner (System.in);


      System.out.println("Enter a number: ");
      int number = scanner.nextInt();


      if (number % 2 == 0) {
         System.out.println(number + " is even.");
      } else {
         System.out.println(number + " is odd.");
      }
       scanner.close();
     }
     }
```



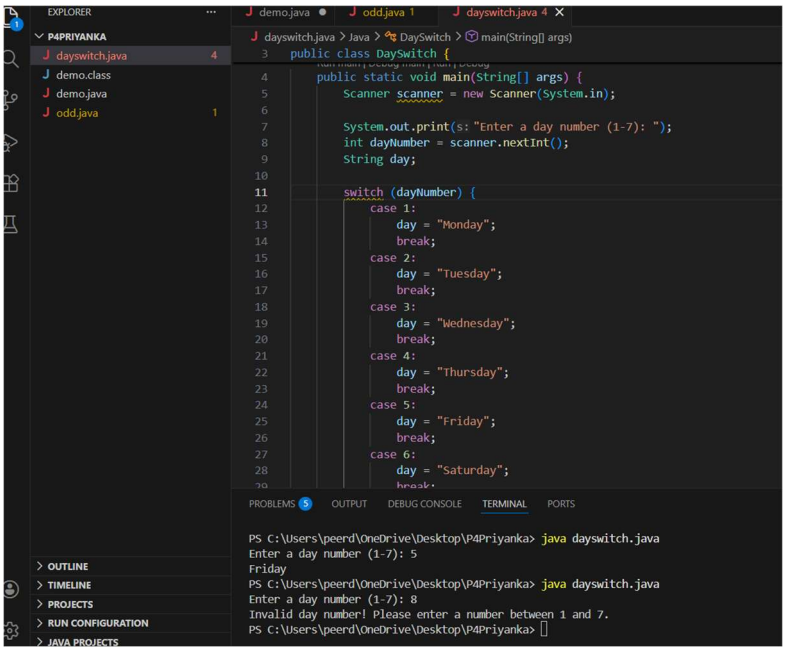
Exercise 2: Switch Statement Write a Java program that takes a day number (1-7) and prints the corresponding day of the week.

Input (Program - dayswitch.java):

Java

```
import java.util.Scanner;


public class DaySwitch {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a day number (1-7): ");
        int dayNumber = scanner.nextInt();
        String day;


        switch (dayNumber) {
            case 1:
                day = "Monday";
```

```
            break;
        case 2:
            day = "Tuesday";
            break;
        case 3:
            day = "Wednesday";
            break;
        case 4:
            day = "Thursday";
            break;
        case 5:
            day = "Friday";
            break;
        case 6:
            day = "Saturday";
            break;
        case 7:
            day = "Sunday";
            break;
        default:
            day = "Invalid day number! Please enter a number between 1 and 7.";
            break;
      }
      System.out.println(day);
       scanner.close();
   }
}
```



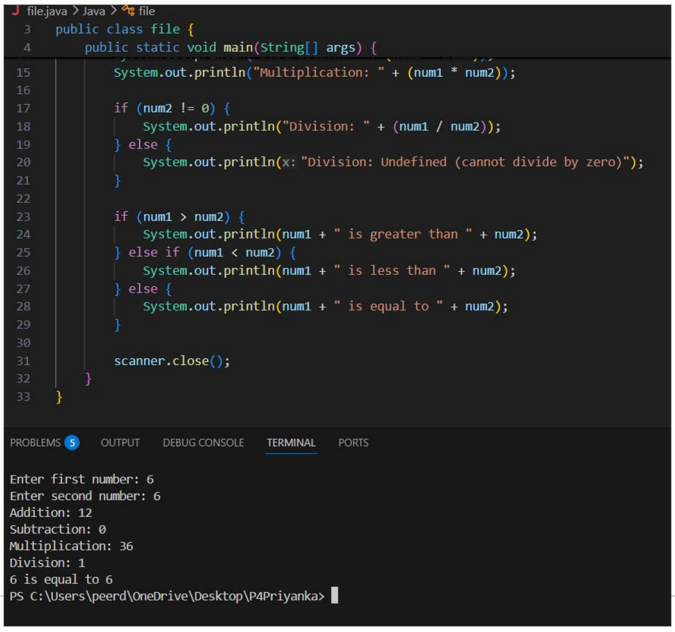
Exercise 3: Arithmetic and Relational operators Write a Java Program that takes two numbers and performs addition, subtraction, multiplication, and division. Also, check if the first number is greater than the second.

Input (Program - Main.java):

Java

```
import java.util.Scanner;

public class Main {

   public static void main(String[] args) {

      Scanner scanner = new Scanner(System.in);

      System.out.print("Enter first number: ");

      int num1 = scanner.nextInt();


      System.out.print("Enter second number: ");

      int num2 = scanner.nextInt();


      System.out.println("Addition: " + (num1 + num2));
```

```
        System.out.println("Subtraction: " + (num1 - num2));
        System.out.println("Multiplication: " + (num1 * num2));
        if (num2 != 0) {
            System.out.println("Division: " + (num1 / num2));
        } else {
            System.out.println("Division: Undefined (cannot divide by zero)");
        }

        if (num1 > num2) {
            System.out.println(num1 + " is greater than " + num2);
        } else if (num1 < num2) {
            System.out.println(num1 + " is less than " + num2);
        } else {
            System.out.println(num1 + " is equal to " + num2);
        }
         scanner.close();
    }
}
```

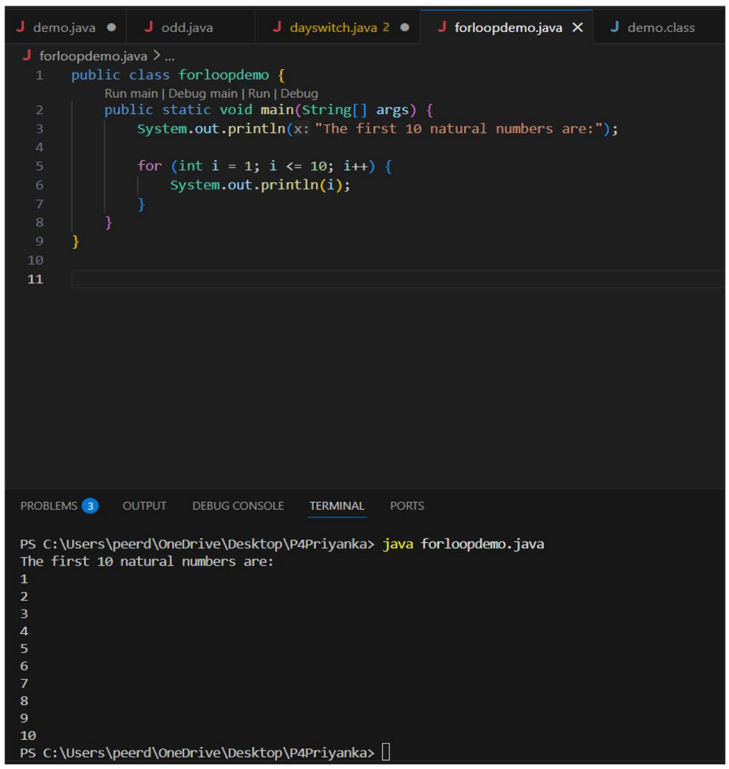


Exercise 4: For Loop Write a Java Program that prints the first 10 natural numbers using a for loop.

Input (Program - ForLoopDemo.java):

Java

```
public class ForLoopDemo {
   public static void main(String[] args) {
      System.out.println("The first 10 natural numbers are:");

      for (int i = 1; i <= 10; i++) {
         System.out.println(i);
      }
   }
}
```

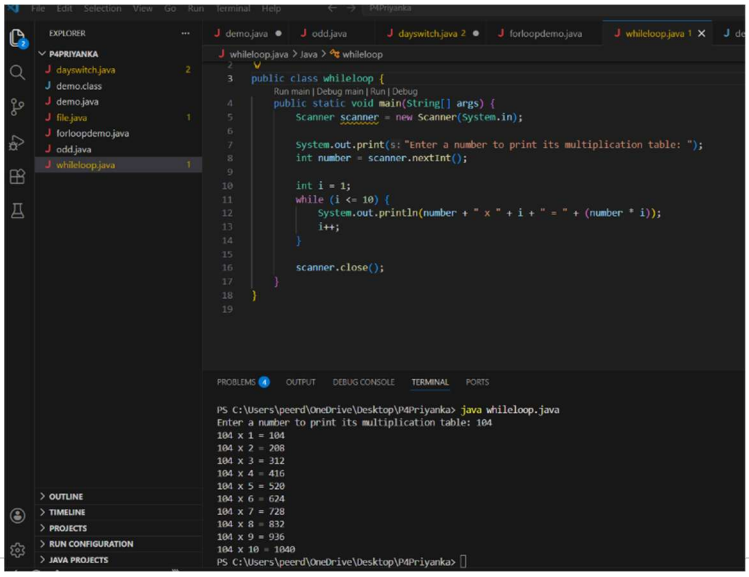


Exercise 5: While Loop Write a Java Program that takes a number from the user and prints the multiplication table of that number using a While loop.

Input (Program - WhileLoopDemo.java):

Java

```
import java.util.Scanner;

public class WhileLoopDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number to print its multiplication table: ");
        int number = scanner.nextInt();
        int i = 1;
        while (i <= 10) {
            System.out.println(number + " x " + i + " = " + (number * i));
            i++;
        }
        scanner.close();
    }
}
```



PRACTICAL-3

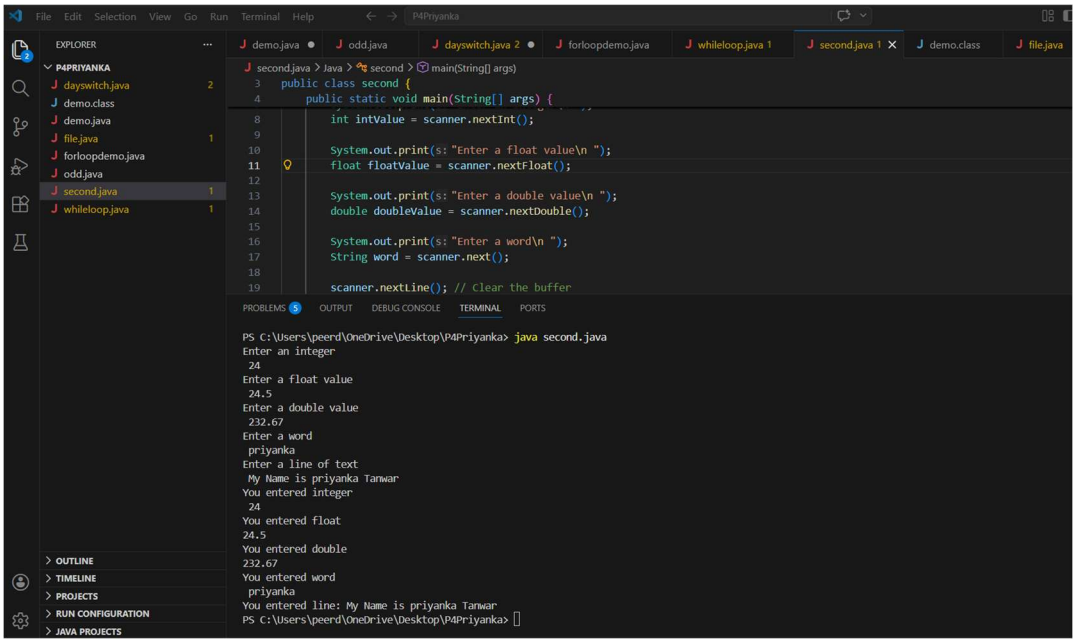
Aim: Program to scan the input using scanner class

Theory: Methods commonly used in the scanner class:

1. nextln ()- Read an integer.
2. nextDouble ()- Read a double.
3. nextLine ()- Reads a string (including spaces).
4. next ()- Read a Single word (upto a space).
5. nextBoolean ()- Reads a Boolean (true or flase).

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      System.out.print("Enter an integer\n ");
      int intValue = scanner.nextInt();

      System.out.print("Enter a float value\n ");
      float floatValue = scanner.nextFloat();

      System.out.print("Enter a double value\n ");
      double doubleValue = scanner.nextDouble();

      System.out.print("Enter a word\n ");
      String word = scanner.next();
```

```
        scanner.nextLine(); // Clear the buffer


        System.out.print("Enter a line of text\n ");
        String line = scanner.nextLine();


        System.out.println("You entered integer\n " + intValue);
        System.out.println("You entered float\n" + floatValue);
        System.out.println("You entered double\n" + doubleValue);
        System.out.println("You entered word\n " + word);
        System.out.println("You entered line: " + line);


        scanner.close();
    }
}
```



PRACTICAL-4

Aim: Programming exercise on arrays and functions in Java.

Theory:

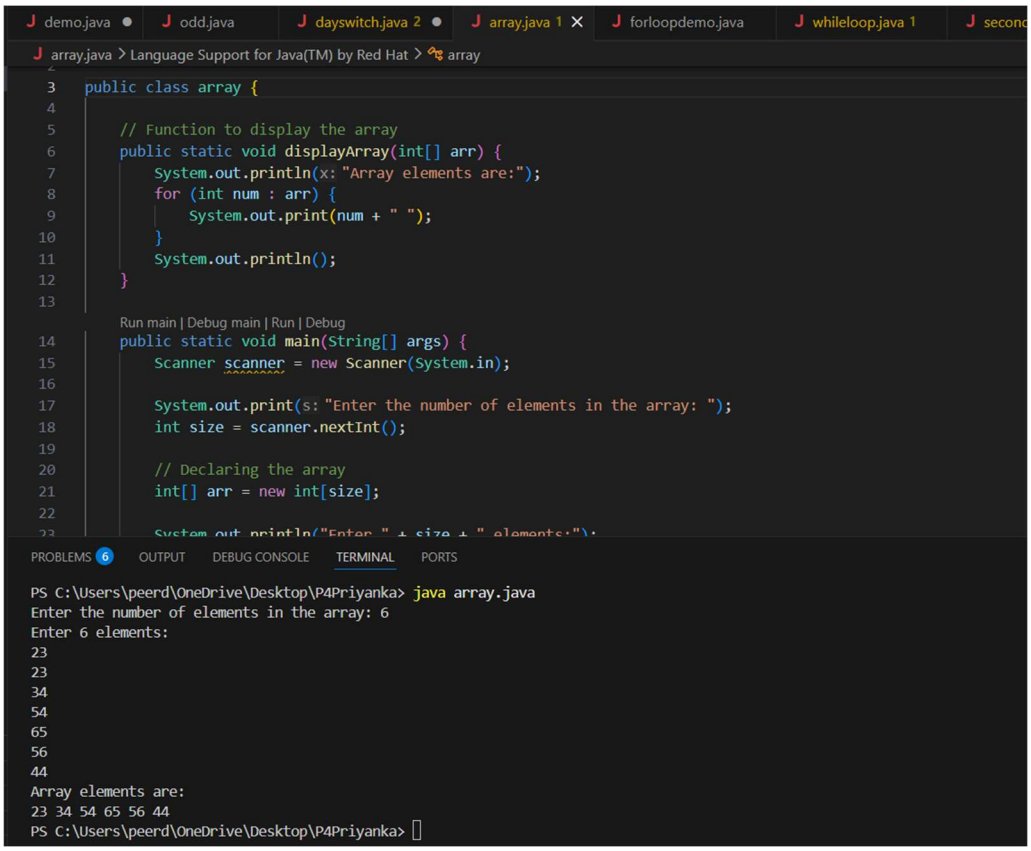
1. Arrays: An array is a collection of elements of the same type, stored in contiguous memory locations. It allows you to store multiple values in a single variable.
2. Functions: Functions are blocks of code that perform a specific task and can be reused throughout the program.

Input (Program - Main.java):

Java

```
import java.util.Scanner;


public class Main {
    // Function to display the array
   public static void displayArray(int[] arr) {
      System.out.println("Array elements are:");
      for (int num: arr) {
         System.out.print(num + " ");
      }
      System.out.println();
   }


   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);


      System.out.print("Enter the number of elements in the array: ");
      int size = scanner.nextInt();
```

```
        // Declaring the array
        int[] arr = new int[size];

        System.out.println("Enter " + size + " elements:");
        for (int i = 0; i < size; i++) {
            arr[i] = scanner.nextInt();
        }

        // Calling the function to print the array
        displayArray(arr);

        scanner.close();
    }
}
```



PRACTICAL NO. 5

Aim: Program to demonstrate the concept of classes and objects using access specifiers.

Theory:

1. **Class:** A blueprint for creating objects. It defines properties and methods that the objects created from the class can have.
2. **Object:** An instance of a class. It represents a specific entity with the properties and behaviours defined by the class.
3. **Access Specifiers:**
    - **Public:** Members declared as public can be accessed from any other class.
    - **Private:** Members declared as private can only be accessed within the same class.
    - **Protected:** Members declared as protected can be accessed within the same package and by subclasses.

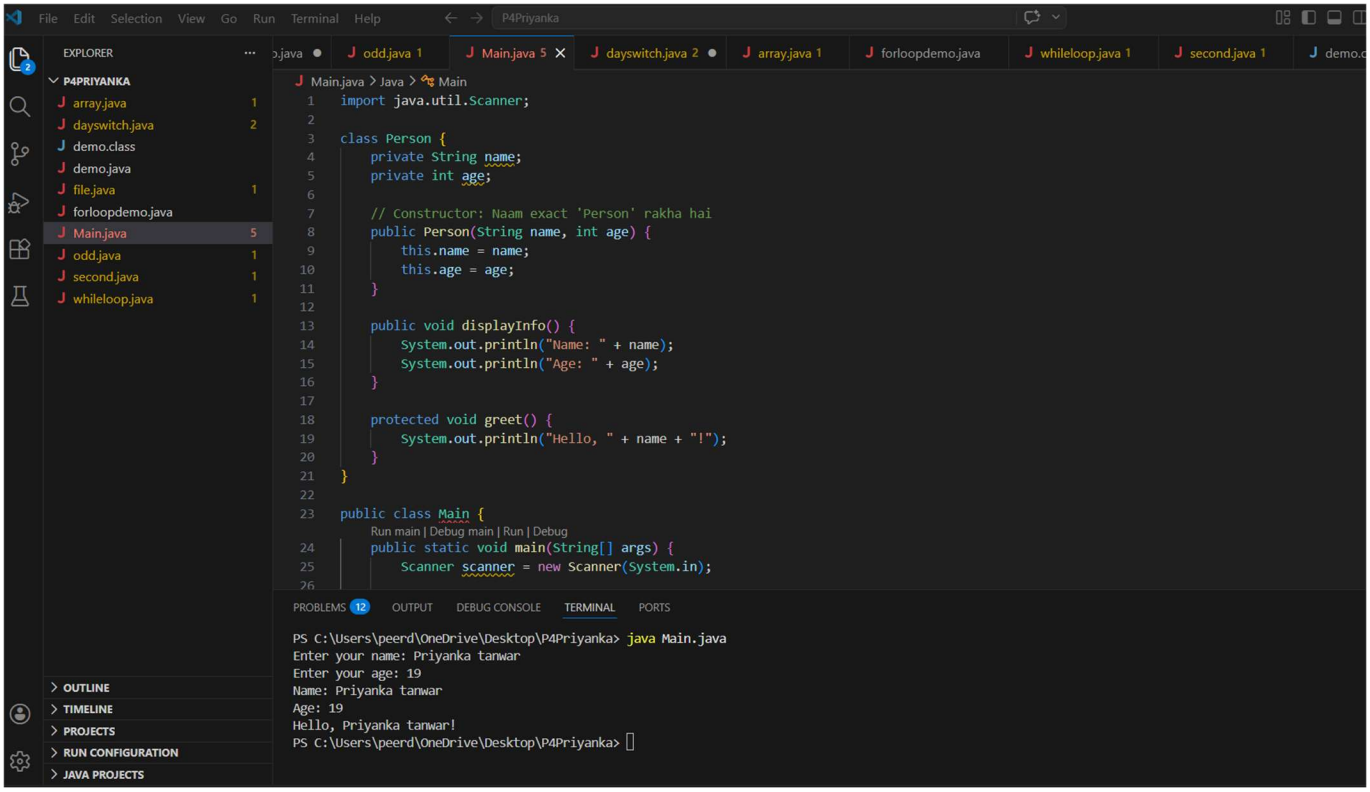
Input (Program - Main.java):

Java

```
import java.util.Scanner;


class Person {
   // Private variables: Only accessible within this class
   private String name;
   private int age;


   // Public constructor: Accessible from anywhere to create objects
   public Person(String name, int age) {
      this.name = name;
      this.age = age;
   }
```

```
    // Protected method: Accessible within the same package or subclasses
    protected void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }

    // Public method: Accessible from anywhere, used to access private variables safely
    public void introduce() {
        System.out.println("Hello, let me introduce myself.");
        displayDetails(); // Calling the protected method
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("--- Entering Details ---");
        System.out.print("Enter name: ");
        String username = scanner.nextLine();

        System.out.print("Enter age: ");
        int userAge = scanner.nextInt();

        // Creating an Object of the Person class
        Person p1 = new Person(userName, userAge);
```

```
        System.out.println("\n--- Displaying Details ---");
        // Calling a public method to trigger the output
        p1.introduce();

        scanner.close();
    }
}
```



PRACTICAL-6

Aim: Program to demonstrate the use of constructors in Java.

Theory: A constructor is a special method that is called when an object is instantiated. It has the same name as the class and does not have a return type. Constructors are used to initialize the object's attributes.

Features of constructors:

1. Same name as the class.
2. No return type, not even void.

Types of constructors in Java:

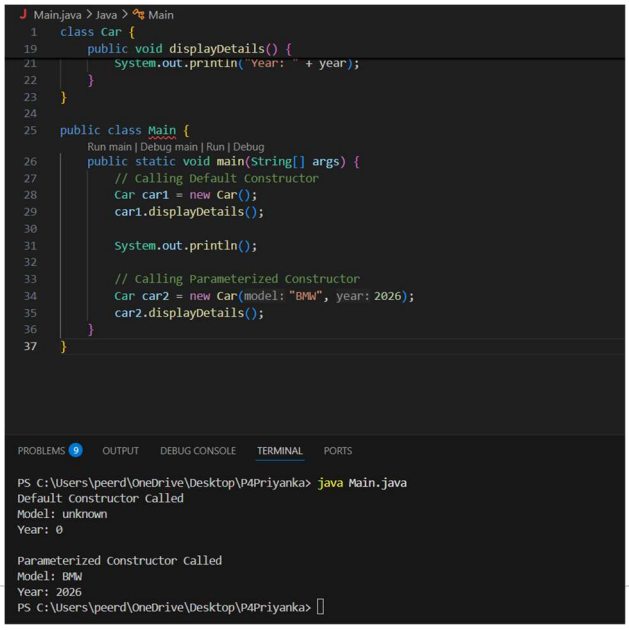
1. **Default Constructor:** Provided by the compiler or defined explicitly without parameters.
2. **Parameterized Constructor:** Accepts arguments to initialize the object with specific values.
3. **Copy Constructor:** Creates a new object by copying values from another object.

Input (Program - Main.java):

Java

```
class Car {
   String model;
   int year;
   // Default Constructor
   public Car () {
      model = "unknown";
      year = 0;
      System.out.println("Default Constructor Called");
   }
   // Parameterized Constructor
   public Car (String model, int year) {
      this. model = model;
```

```
        this.year = year;
        System.out.println("Parameterized Constructor Called");
    }
    public void displayDetails () {
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
}
public class Main {
    public static void main (String [] args) {
        // Calling Default Constructor
        Car car1 = new Car ();
        car1.displayDetails();
        System.out.println();
       // Calling Parameterized Constructor
        Car car2 = new Car ("BMW", 2026);
        car2.displayDetails();
    }
}
```



PRACTICAL NO. 7

Aim: Programming exercise on different types of inheritance in Java.

Theory: Inheritance is a mechanism in Java where one class (subclass or child class) inherits the properties and behaviours (methods) of another class (superclass or parent class). This promotes code reusability and establishes a relationship between classes.

Types of Inheritance demonstrated:

1. **Single Inheritance:** A class inherits from one superclass (e.g., Dog inherits from Animal).
2. **Multi-level Inheritance:** A class inherits from a subclass, forming a chain (e.g., Puppy inherits from Dog, which inherits from Animal).
3. **Hierarchical Inheritance:** Multiple classes inherit from a single superclass (e.g., both Dog and Cat inherit from Animal).
4. **Multiple Inheritance (via Interfaces):** A class implements more than one interface (e.g., Duck implements both Bird and Fish).

Input (Program - Main.java):

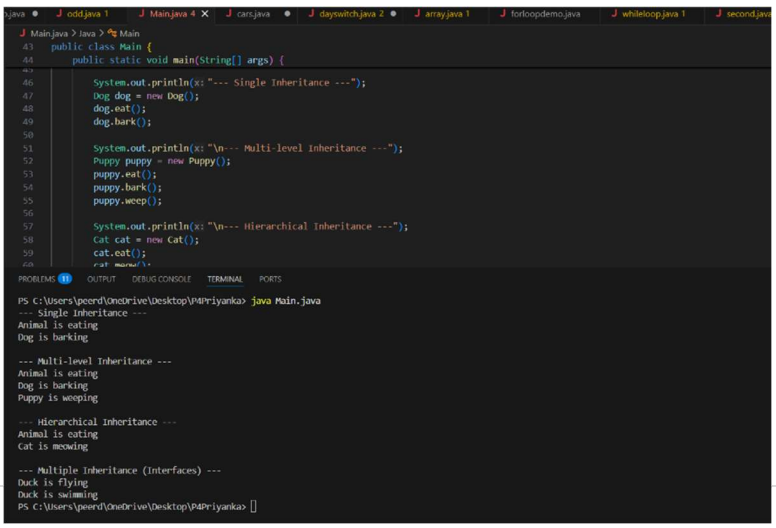
Java

```
class Animal {
   void eat () {
      System.out.println("Animal is eating");
   }
}
```

```
class Dog extends Animal {
    void bark () {
        System.out.println("Dog is barking");
    }
}

class Puppy extends Dog {
    void weep () {
        System.out.println("Puppy is weeping");
    }
}

class Cat extends Animal {
    void meow () {
        System.out.println("Cat is meowing");
    }
}
interface Bird {
    void fly ();
```

```
}
interface Fish {
    void swim ();
}
class Duck implements Bird, Fish {
    public void fly () {
        System.out.println("Duck is flying");
    }
public void swim () {
        System.out.println("Duck is swimming");
    }
}
public class Main {
    public static void main (String [] args) {
        System.out.println("--- Single Inheritance ---");
        Dog dog = new Dog ();
        dog.eat ();
        dog. bark();
       System.out.println("\n--- Multi-level Inheritance ---");
```

```
        Puppy puppy = new Puppy ();

        puppy.eat ();

        puppy. bark ();

        puppy. weep ();

        System.out.println("\n--- Hierarchical Inheritance ---");

        Cat cat = new Cat ();

        cat.eat ();

        cat.meow();

        System.out.println("\n--- Multiple Inheritance (Interfaces) ---");

        Duck duck = new Duck ();

        duck.fly ();

        duck. swim ();

    }

}
```



PRACTICAL NO. 8

Aim: Program to demonstrate the concept of method overloading and overriding.

Theory:

1. **Method Overloading:** This occurs when multiple methods in the same class have the same name but different parameters (different type, number, or both). It allows methods to perform similar functions with different types of input.
2. **Method Overriding:** This occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. It allows a subclass to provide a specific behaviour for a method that is already defined in its parent class.

Input (Program - Main.java):

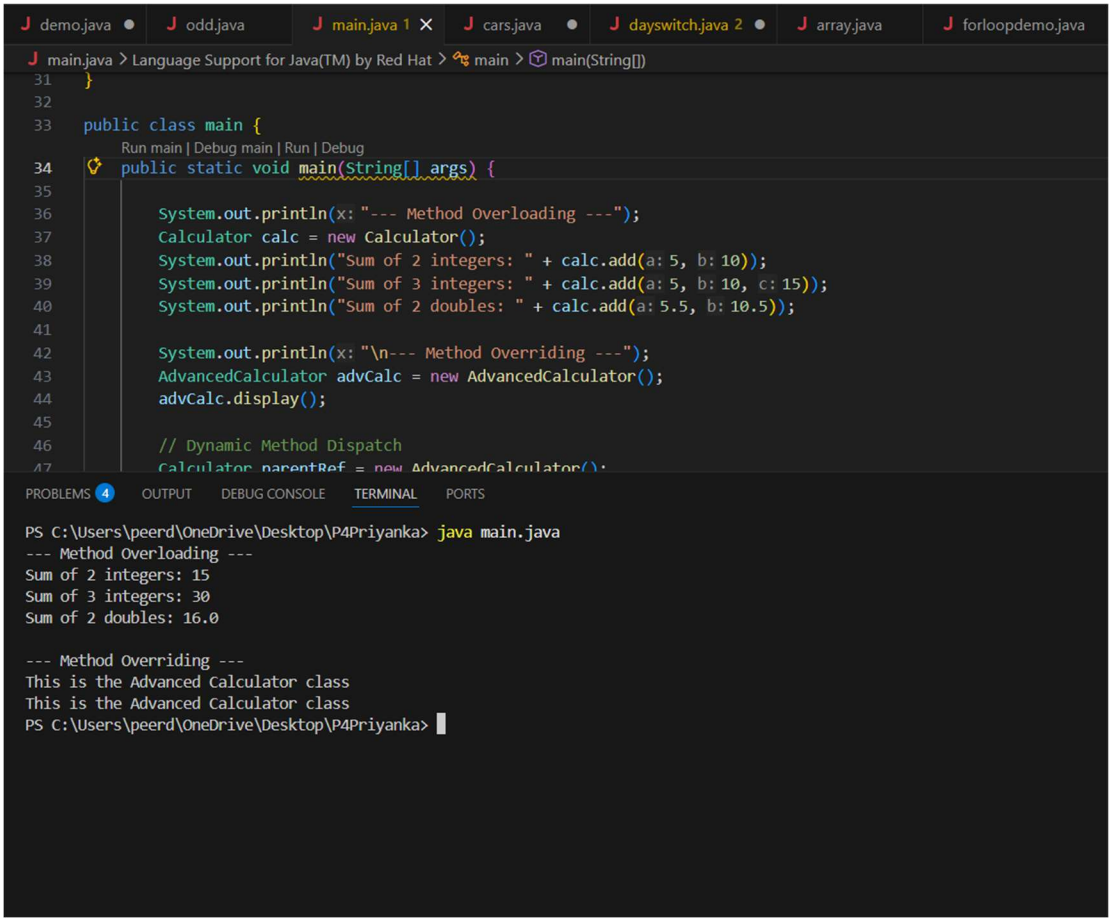
Java

```
class Calculator {


  // Overloaded method 1: Two integer parameters
  int add (int a, int b) {
    return a + b;
  }


  // Overloaded method 2: Three integer parameters
  int add (int a, int b, int c) {
    return a + b + c;
  }


  // Overloaded method 3: Two double parameters
  double add (double a, double b) {
    return a + b;
```

```
    }

    // Method to be overridden
    void display () {
        System.out.println("This is the Calculator class");
    }
}

class AdvancedCalculator extends Calculator {

    // Overriding the display method from the parent class
    @Override
    void display () {
        System.out.println("This is the Advanced Calculator class");
    }
}

public class Main {
    public static void main (String [] args) {

        System.out.println("--- Method Overloading ---");
        Calculator calc = new Calculator ();
        System.out.println("Sum of 2 integers: " + calc.add (5, 10));
        System.out.println("Sum of 3 integers: " + calc.add (5, 10, 15));
        System.out.println("Sum of 2 doubles: " + calc.add (5.5, 10.5));

        System.out.println("\n--- Method Overriding ---");
```

```
        AdvancedCalculator advCalc = new AdvancedCalculator ();
        advCalc.display();

        // Dynamic Method Dispatch
        Calculator parentRef = new AdvancedCalculator ();
        parentRef.display();
    }
}
```

Output:



PRACTICAL NO. 9

Aim: Program to demonstrate the concept of packages, abstract classes, and interfaces.

Theory:

1. **Packages:** A package is a namespace that organizes a set of related classes and interfaces. Conceptually similar to directories on your computer, packages help avoid name conflicts and can control access with protected and default access levels.
2. **Abstract Classes:** An abstract class is a class that cannot be instantiated on its own and may contain abstract methods (methods without a body). Subclasses must provide implementations for these abstract methods.
3. **Interfaces:** An interface is a reference type in Java that is similar to a class but can only contain constants, method signatures, default methods, static methods, and nested types. Interfaces cannot contain instance fields. A class implements an interface, providing the body for all the methods defined in the interface.

Input (Program - Main.java):

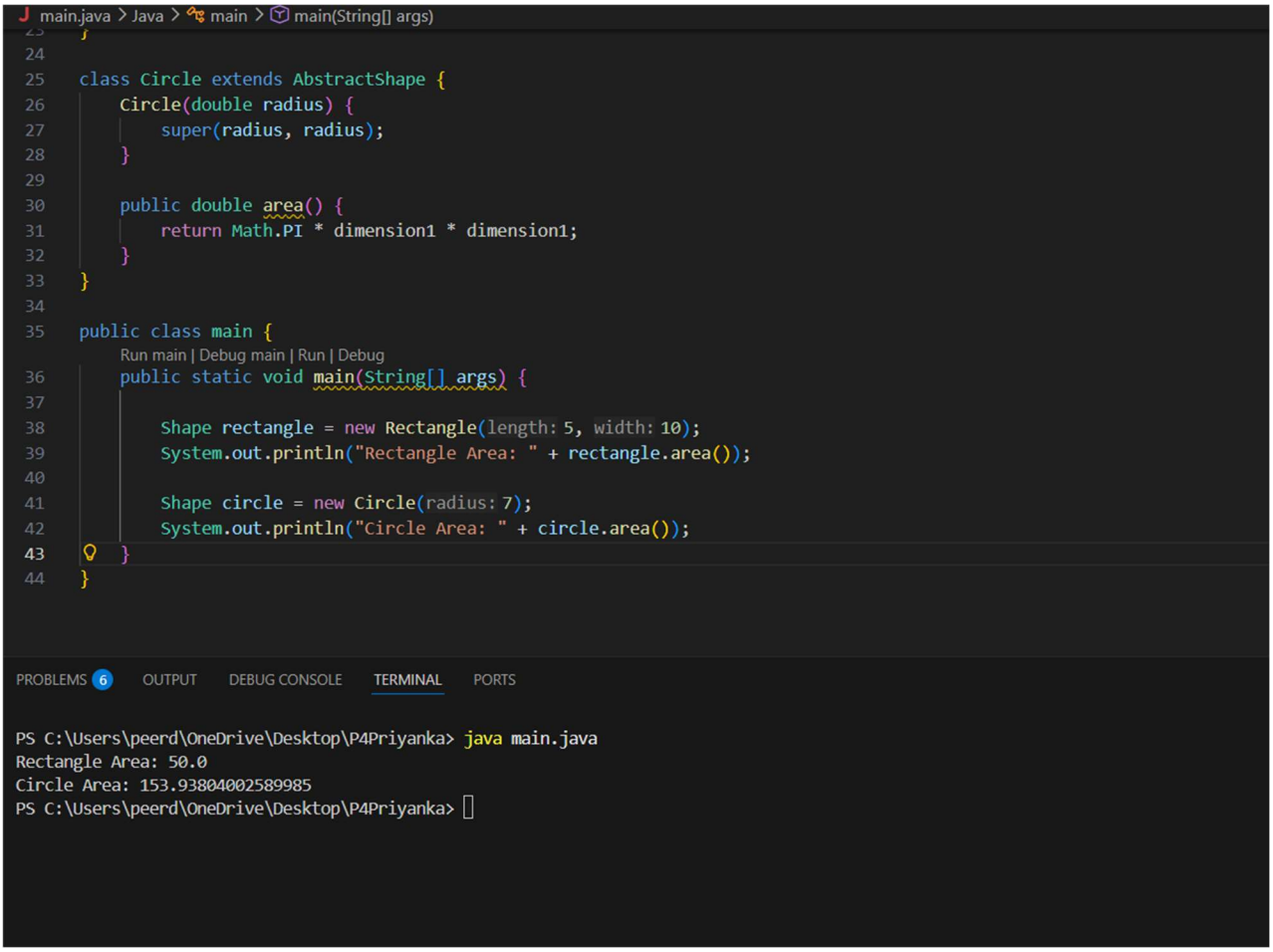
Java

```
interface Shape {
   double area ();
}


abstract class AbstractShape implements Shape {
   double dimension1;
   double dimension2;


   AbstractShape (double d1, double d2) {
      this. dimension1 = d1;
      this. dimension2 = d2;
```

```
    }
}

class Rectangle extends AbstractShape {
    Rectangle (double length, double width) {
        super (length, width);
    }

    public double area () {
        return dimension1 * dimension2;
    }
}

class Circle extends AbstractShape {
    Circle (double radius) {
        super (radius, radius);
    }

    public double area() {
        return Math.PI * dimension1 * dimension1;
    }
}

public class Main {
    public static void main(String[] args) {

        Shape rectangle = new Rectangle (5, 10);
```

```
        System.out.println("Rectangle Area: " + rectangle.area());


        Shape circle = new Circle (7);
        System.out.println("Circle Area: " + circle. area ());
    }
}
```

OUTPUT:



PRACTICAL NO. 10

Aim: Programming exercise on exception handling.

Theory:

1. Exception: An exception is an event that disrupts the normal flow of a program's execution. It can occur due to various reasons, such as invalid input, file not found, or network issues.
2. Try – Catch Block: A try – catch block is used to handle exceptions. The code that might throw an exception is placed inside the try block, and the handling code is placed inside the catch block.
3. Finally Block: The finally block is used to execute code after the try-catch block, regardless of whether an exception was thrown or not. It is typically used for cleanup activities.
4. Throwing Exceptions: You can throw exceptions manually using the throw keyword.

Input (Program - Main.java):

Java

```
import java. util. Scanner;


public class Main {
   public static void main (String [] args) {
      Scanner scanner = new Scanner(System.in);


      try {
         System.out.print("Enter the first number: ");
         int num1 = scanner. nextInt ();


         System.out.print("Enter the second number: ");
```

```
        int num2 = scanner. nextInt ();

        int result = num1 / num2;
        System.out.println("Result: " + result);

    } catch (ArithmeticException e) {
        System.out.println("Error: Cannot divide by zero!");

    } catch (java.util.InputMismatchException e) {
        System.out.println("Error: Invalid input. Please enter integers only.");

    } finally {
        System.out.println("Program execution completed.");
        scanner. close ();
    }
  }
}
```

OUTPUT: